Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S03: Hands-On Lab Walkthrough

Developing High-Throughput Idempotent Producers with Snappy Compression

Lab Objectives

In this hands-on terminal lab, you will:
1. Configure and run an enterprise-grade Kafka producer using `confluent-kafka` in Python.
2. Observe dual-threaded non-blocking behavior using asynchronous send calls and delivery callbacks.
3. Tune batching parameters (`batch.size` , `linger.ms`) and evaluate network throughput improvements.
4. Verify End-to-End Snappy compression on live event batches.
5. Prove mathematically that Idempotence eliminates duplicate messages during network retries.

Step 1: Pre-Flight Environment Check

Open your terminal and navigate to the lab root:

```
cd "${LAB_ROOT:-delivery_materials/00_Lab}" 2>/dev/null || cd 00_Lab || cd ../../00_Lab
```

Verify that the virtual environment and cluster are operational:

```
tools/check_stack.sh
```

Ensure all 6 services report `PASS` .

Step 2: Create the Orders Topic

Create the retail orders topic with 3 partitions and a replication factor of 3:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka1:9092 \
  --create --topic nile.retail.orders \
  --partitions 3 --replication-factor 3 \
  --config min.insync.replicas=2
```

Step 3: Run the Consumer in Listening Mode

Open a secondary terminal window to monitor message arrival across partitions:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server kafka1:9092 \
  --topic nile.retail.orders \
  --property print.key=true \
  --property print.partition=true \
  --property print.offset=true
```

Step 4: Inspect the Python Producer Script

Inspect the production-ready code in `python/s03_producer.py`:

```
cat python/s03_producer.py
```

Notice the configuration dictionary:

```
conf = {
    "bootstrap.servers": "localhost:9092",
    "acks": "all",
    "enable.idempotence": True,
    "linger.ms": 10,
    "batch.size": 32768,
    "compression.type": "snappy",
    "max.in.flight.requests.per.connection": 5
}
```

Step 5: Execute the Producer Pipeline

Run the producer to emit 500 validated orders:

```
python3 -m python.s03_producer --acks all --idempotent --orders 500 --compression snappy
```

Expected Console Output:

```
sent=500 delivered=500 failed=0 skipped_invalid=0
```

In your secondary consumer terminal, observe that:

- Messages are evenly distributed across Partitions 0, 1, and 2.
- Records with identical order keys (`01` , `02`) consistently land on the exact same partition.
- Each delivery callback returns immediately upon broker ISR commit.

Step 6: Verify Idempotent Deduplication (Unit Test Gate)

Run the automated idempotent validation test suite:

```
.venv/bin/pytest tests/test_s03_producer_logic.py -v
```

Expected Output:

```
tests/test_s03_producer_logic.py::test_valid_event_becomes_key_and_value PASSED [ 10%]
tests/test_s03_producer_logic.py::test_same_order_gives_same_key PASSED  [ 20%]
tests/test_s03_producer_logic.py::test_missing_order_id_is_rejected PASSED [ 30%]
tests/test_s03_producer_logic.py::test_bad_amount_is_rejected[0] PASSED  [ 40%]
tests/test_s03_producer_logic.py::test_bad_amount_is_rejected[-5] PASSED [ 50%]
tests/test_s03_producer_logic.py::test_bad_amount_is_rejected[None] PASSED [ 60%]
tests/test_s03_producer_logic.py::test_unknown_type_is_rejected PASSED   [ 70%]
tests/test_s03_producer_logic.py::test_idempotent_broker_accepts_first_write PASSED [ 80%]
tests/test_s03_producer_logic.py::test_idempotent_broker_rejects_retry_duplicate PASSED [ 9
0%]
tests/test_s03_producer_logic.py::test_idempotent_broker_rejects_out_of_order_sequence PASSE
D [100%]
============================== 10 passed in 0.03s ==============================
```

Step 7: Run the S03 Automated Verification Script

Run the automated verification gate to validate your workstation results:

```
../delivery_materials/S03_Producer_Internals_Delivery_Semantics/Trainer/verify_s03.sh
```

Verify that all 5 tests report `PASS` and finish with:
`★ PASS: S03 Lab Requirements 100% Satisfied! ★`

🌟 Gold Stretch Challenge: Compression Efficiency Benchmark

Distinguished Engineer Challenge: If you complete the core lab before time expires, execute this compression efficiency benchmark:

1. Measure the compression ratio and throughput impact across different algorithms (`none` , `gzip` , `snappy` , `lz4` , `zstd`):

- Publish 20,000 JSON records using the standard performance harness:

`bash`

```
   docker exec -it kafka1 /opt/kafka/bin/kafka-producer-perf-test.sh      --topic retail.orders      --num-records 20000      --record-size 1024      --throughput -1      --producer-props bootstrap.servers=kafka1:9092 acks=all compression.type=snappy
```

2. Compare the output metrics: Records/sec, MB/sec throughput, 95th percentile latency, and 99th percentile latency.